After our `stackLayout` function processes our dataset, we can get the results by running `stackLayout(stackData)`. The layout creates `x`, `y`, and `y0` functions corresponding to the top and bottom of the object at the x position. If we use the stack layout to create a streamgraph, then it requires a corresponding area generator:

```
var stackArea = d3.svg.area()
.x(function(d) { return xScale(d.x); })
.y0(function(d) { return yScale(d.y0); })
.y1(function(d) { return yScale(d.y0 + d.y); });
```

Usually at some point you need to pass the data to a scale function to fit it to the screen.

After we have our data, layout, and area generator in order, we can call them all as part of the selection and binding process. This gives a set of SVG `<path>` elements the necessary shapes to make our chart:

```
d3.select("svg").selectAll("path")
   .data(stackLayout(stackData))
   .enter()
   .append("path")
   .style("fill", function(d) {return movieColors(d.name);})
   .attr("d", function(d) { return stackArea(d.values); });
```

The data being bound is stackData processed by stackLayout().

A color scale that associates a unique color with each object in the array

The area generator takes the values from our data processed by the layout to get the SVG drawing code.

The result, as shown in figure 5.20, isn't a streamgraph but rather a stacked area chart, which isn't that different from a streamgraph, as you'll soon find out.

The stack layout has an `.offset()` function that determines the relative positions of the areas that make up the chart. Although we can write our own offset functions to create exotic charts, this function recognizes a few keywords that achieve the typical effects we're looking for. We'll use the `silhouette` keyword, which centers the drawing

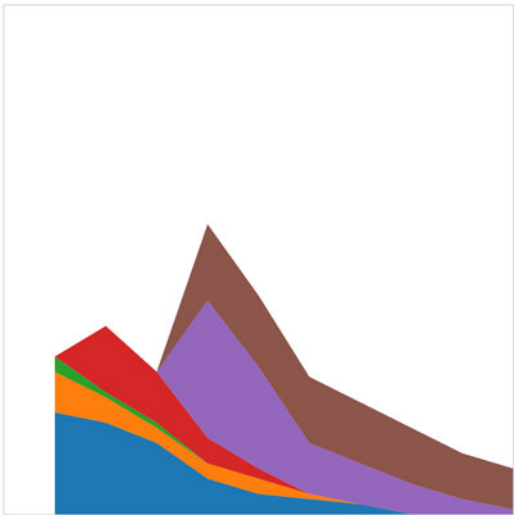


Figure 5.20 The stack layout default settings, when tied to an area generator, produce a stacked area chart like this one.